



Star-Prime

Trouble Ticket Management System

Group 12

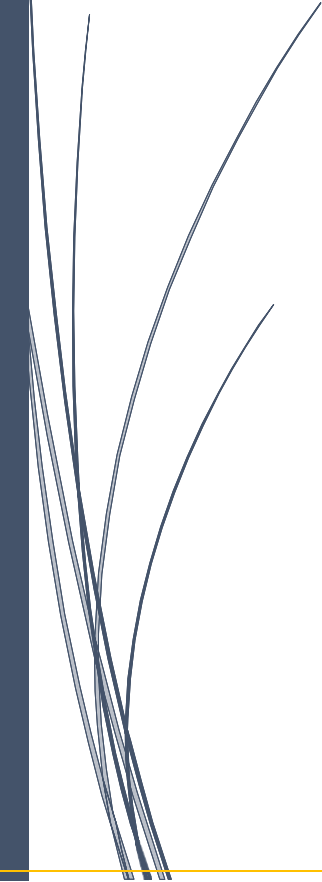
Shashank Chandra

Sakshi Santosh Sonawane

Vincent Sangala

Nurul Chowdhury

Yi Hen, Yang



Index

1. Introduction
2. Implementation
3. Database Design
4. Complex Queries
5. Stored Procedures
6. Functions
7. Triggers

Introduction

In the past, maintenance requests in residential buildings were handled manually—via phone calls and paper logs. This outdated method required significant time and effort.

Our project, **Star-Prime**, aims to automate these processes, enabling residents and managers to handle repairs more efficiently through a centralized digital platform.

Estates and residential buildings have evolved over the years to commit to customer satisfaction and enable service availability and its essential to maintain a system that can handle maintenance requests. In the older days the offices used to handle phone calls and record issues on a piece of paper which was later process by office clerks who facilitated service requests. This resulted in abundant manual processing and effort which can be reduced by automating service line requests., the offices used to handle phone calls and record issues on a piece of paper, which was later processed

In our Project Star-Prime we avoid manual processing which facilitates in the smooth functioning of maintenance and repairs that increases customer satisfaction while reducing ancient mechanisms of building management. Assuming we have a residential society star-prime is an all-in-one solution to all the actors in the frame to communicate and facilitate repairs and maintenance without having to pick phone calls.

This involves raising tickets with other related and required information that will help the repairmen to plan for the repair which will result in less time used for implementing the repair works. Additionally efficient data collection can also be used for analytical purposes that can used by the business to improve service quality and provide additional features that can bring positive impact to the community and living standards.

Project Scope and Vision:

Provide a platform and framework that concentrates on the repairs and services that occur in a home community. With existing applications partially completing the said requirements with a prospect for analytics, Star-Prime integrates different aspects of entities within a community to a single application which facilitates all functions under a single umbrella a.k.a ONE-TECHNOLOGY.

Target audience

The audience encompasses a diverse range of users, tenants, Owners, managers, and service technicians. These users have a wide variety of functions in view of managing the application or using it to raise tickets. Users will be managed with the help of access groups and security permission considering valid authorization and authentication.

Features for end user

Star Prime offers an easy-to-use platform where users can raise tickets, track their status, and get real-time updates. Users receive alerts, add comments, and upload attachments for better issue resolution. The system ensures quick support with clear priorities, SLA timelines, and access to ticket history all accessible anytime from any device.

Constraints

We used primary and foreign keys for relational integrity, unique and NOT NULL constraints to ensure data validity, and ENUMs to control input values. Triggers were added to automate SLA validation and logging. These ensure reliable and consistent data across the system.

Risks

Potential risks include data breaches if user credentials aren't securely managed, SLA violations if automation fails, and performance issues with large datasets. There's also a risk of incorrect escalation if rules are misconfigured. Regular audits and validations help mitigate these.

Implementation

The implementation of the StarPrime database system involved a structured approach to designing and building a scalable and efficient solution to manage property operations and ticketing workflows.

We began by identifying all the core entities involved in the system, such as users, properties, support tickets, issue categories, and service providers. The relationships among these entities were then carefully mapped to ensure normalization, avoid data redundancy, and maintain data integrity.

Each functional area was designed as a module:

- **User and Role Management:** A flexible role-based access control system was implemented using the Users, User Roles, and Departments tables to support various user types such as tenants, property managers, support agents, and administrators.
- **Property and Issue Management:** The Properties table stores details of all properties, with support for assigning managers. Issue Categories were predefined to standardize the types of requests users can raise.
- **Ticketing System:** The Tickets table is the core of the application. It tracks all service requests, priorities, statuses, SLA deadlines, and assignments. To enhance operational clarity, tickets also support resolutions, user comments, and file attachments.
- **SLA and Escalation Logic:** SLA enforcement was implemented using the SLA Details and escalation rules tables. This enables monitoring of response/resolution deadlines and automatic prioritization/escalation handling.
- **Service Providers and Notifications:** External vendors are tracked using the service providers table. User notifications and change logs are managed via Notification Log and Audit Log respectively, ensuring transparency and traceability.

Database Design.

The StarPrime database is designed to efficiently manage property operations, support ticketing, user roles, service providers, and SLA enforcement. Below is a summary of the major tables and how they relate to one another.

1.Users

Stores information about all users (e.g., tenants, support agents, property managers).

Relationships:

- Linked to User_Roles via role_id
- Linked to Departments via department_id
- Referenced in Tickets (as the person who raised or is assigned the ticket)
- Referenced in Audit_Log, Comments, and Notification_Log

2. User Roles: -

Defines different types of users (e.g., admin, support agent, tenant).

Used to assign role-based access and permissions to users.

3. Departments: -

Organizes users into functional units such as Maintenance, IT, or Security.

4. Properties: -

Stores property details like name, address, and manager.

Relationships:

- Linked to Users through manager_id
- Referenced by Tickets and Maintenance_Requests

5. Issue Categories

Lists types of issues (Plumbing, Electrical, etc.)

Relationships:

- Referenced in Tickets
- Connected to SLA_Details
- Tickets

Central table for reporting and tracking issues.

Stores description, status, priority, SLA deadline, and assigned personnel.

Relationships:

- Linked to Users, Properties, Issue_Categories, Ticket_Status, and optionally Service_Providers
- Referenced by Ticket_Resolutions, Comments, Attachments, Audit_Log, Notification_Log

7. Ticket_Status

Stores status of tickets (e.g., Open, In Progress, Closed).

8. SLA_Details

Defines SLA response, resolution, and escalation times for issue categories.
Helps monitor service performance.

9. Escalation_Rules

Outlines when tickets should be escalated based on priority (Low, Medium, High, Urgent).

10. Service_Providers

Contains information about third-party vendors and contractors.
Referenced in Tickets.

11. Ticket_Resolutions

Stores resolution details for each ticket.
Linked to Tickets.

12. Comments

Stores comments made by users on specific tickets.
Relationships:

- Linked to Tickets and Users

13. Attachments

Stores files (images, PDFs, etc.) attached to tickets.
Referenced using ticket_id.

14. Audit_Log

Captures changes made to tickets (who made the change and what was changed).
Supports audit and compliance tracking.

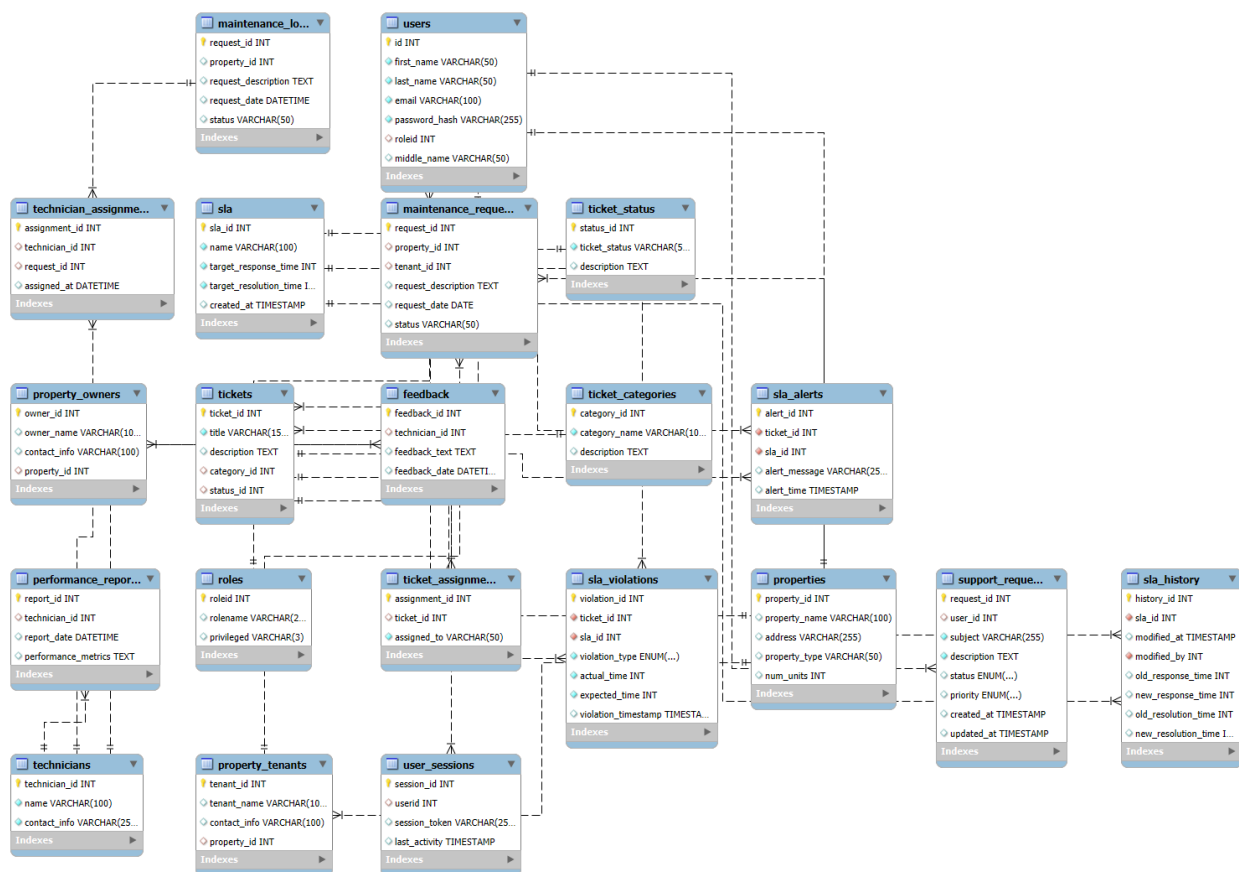
15. Notification_Log

Stores messages and alerts sent to users about ticket updates.
Linked to Users and Tickets.

Overall Design Summary

- The User's table is a central node connected to many modules.
- Tickets serve as the hub for operations, linking to users, properties, categories, statuses, and SLA.
- SLA logic is enforced through SLA_Details and Escalation_Rules.
- Modular support for comments, attachments, audit logs, and notifications ensures traceability and accountability.

ER Diagram



Complex Queries

Query 1 – Detailed Ticket Report

Purpose: Fetch comprehensive ticket information.

Details: Joins multiple tables to show the ticket description, the user who raised it, property name, issue category, ticket status, assigned support agent, and formatted timestamps (created_at and SLA due).

Tables Used: Tickets, Users, Properties, Issue_Categories, Ticket_Status.

ticket_id	description	raised_by	property_name	category_name	status_name	assigned_to	created_at	sla_due
1	Ticket 1: Plumbing issue reported	Alice Johnson	Sunrise Apartments	Plumbing Issue	Open	NULL	2025-05-13 18:44	2025-05-15 11:44
2	Ticket 2: Lift issue reported	Bob Smith	Lakeside Villas	Lift Issue	In Progress	Charlie Brown	2025-05-13 18:44	2025-05-16 11:44
3	Ticket 3: Electrical issue reported	Charlie Brown	Downtown Towers	Electrical Issue	Awaiting Parts	NULL	2025-05-13 18:44	2025-05-14 11:44
4	Ticket 4: HVAC issue reported	Diana Ross	Green Meadows	HVAC Issue	Awaiting Approval	Edward Turner	2025-05-13 18:44	2025-05-15 01:44
5	Ticket 5: Internet issue reported	Edward Turner	Riverfront Residences	Internet Issue	On Hold	NULL	2025-05-13 18:44	2025-05-15 11:44
6	Ticket 6: Garbage collection issue reported	Fiona Lee	Hilltop Homes	Garbage Collection	Escalated	George Martin	2025-05-13 18:44	2025-05-16 11:44
7	Ticket 7: Pest control issue reported	George Martin	Coastal Condos	Pest Control	Assigned	NULL	2025-05-13 18:44	2025-05-14 11:44
8	Ticket 8: Structural issue reported	Hannah Davis	Parkside Estates	Structural Issue	Resolved	Ian Wright	2025-05-13 18:44	2025-05-15 01:44

Query 2 – Ticket Count Per Property with Manager Details

Purpose: Summarize ticket load by property.

Details: Shows each property's ticket count along with the full name of the property manager.

Tables Used: Properties, Tickets, Users.

property_name	manager	ticket_count
Sunrise Apartments	Bob Smith	1
Lakeside Villas	Charlie Brown	1
Downtown Towers	Diana Ross	1
Green Meadows	Edward Turner	1
Riverfront Residences	Fiona Lee	1
Hilltop Homes	George Martin	1
Coastal Condos	Hannah Davis	1
Parkside Estates	Ian Wright	1
Urban Lofts	Julia Roberts	1

Query 3 – Tickets Nearing SLA Breach

Purpose: Identify high-priority tickets.

Details: Lists tickets with less than 12 hours remaining before SLA breach, using a custom function f_timeRemainingForSLA.

Tables Used: Tickets, Ticket_Status.

ticket_id	description	hours_remaining	status_name	sla_due
21	Ticket Jan: plumbing drip in Apt 2B	2939	Closed	2025-01-11 08:00
27	Ticket Jan: plumbing drip in Apt 2B	2939	Closed	2025-01-11 08:00
22	Ticket Feb: Internet outage floor 4	2093	Closed	2025-02-15 14:30
28	Ticket Feb: Internet outage floor 4	2093	Open	2025-02-15 14:30
23	Ticket Mar: elevator stall at lobby	1255	Closed	2025-03-22 12:15
29	Ticket Mar: elevator stall at lobby	1255	Escalated	2025-03-22 12:15
24	Ticket Apr: pest control needed unit 8C	559	Closed	2025-04-20 11:45
30	Ticket Apr: pest control needed unit 8C	559	Awaiting Parts	2025-04-20 11:45

Query 4 – Average Resolution Time per Issue Category

Purpose: Analyze service performance by category.

Details: Calculates average resolution time (in hours) for resolved tickets in each issue category.

Tables Used: Tickets, Ticket_Resolutions, Issue_Categories.

ticket_id	description	hours_remaining	status_name	sla_due
21	Ticket Jan: plumbing drip in Apt 2B	2939	Closed	2025-01-11 08:00
27	Ticket Jan: plumbing drip in Apt 2B	2939	Closed	2025-01-11 08:00
22	Ticket Feb: Internet outage floor 4	2093	Closed	2025-02-15 14:30
28	Ticket Feb: Internet outage floor 4	2093	Open	2025-02-15 14:30
23	Ticket Mar: elevator stall at lobby	1255	Closed	2025-03-22 12:15
29	Ticket Mar: elevator stall at lobby	1255	Escalated	2025-03-22 12:15
24	Ticket Apr: pest control needed unit 8C	559	Closed	2025-04-20 11:45
30	Ticket Apr: pest control needed unit 8C	559	Awaiting Parts	2025-04-20 11:45

Query 5 – Audit History for a Specific Ticket

Purpose: Track changes made to a ticket.

Details: Lists audit logs for a specific ticket, including who made the change, what was changed, and when.

Tables Used: Audit_Log, Users.

Query 6 – Tickets Count by Status

Purpose: Show ticket distribution by status.

Details: Aggregates the total number of tickets per status (e.g., Open, Closed, In Progress).

Tables Used: Tickets, Ticket_Status.

status_name	ticket_count
Open	1
In Progress	1
Awaiting Parts	1
Awaiting Approval	1
On Hold	1
Escalated	1
Assigned	1
Resolved	1
Closed	1

Query 7 – Comments with Ticket and User Details

Purpose: Provide context on ticket discussions.

Details: Lists comments with related ticket descriptions and the full name of the user who posted the comment.

Result Grid						
Filter Rows:						
Export:						
Wrap Cell Content:						
comment_id	ticket_id	ticket_description	user_id	full_name	comment_text	comment_date
21	16	Ticket 16: Door issue reported	1	Alice Johnson	Technician dispatched to fix drip.	2025-01-10 08:15
27	22	Ticket Feb: Internet outage floor 4	1	Alice Johnson	Technician dispatched to fix drip.	2025-01-10 08:15
22	17	Ticket 17: Parking issue reported	2	Bob Smith	ISP case opened; ETA 1 hour.	2025-02-14 14:45
28	23	Ticket Mar: elevator stall at lobby	2	Bob Smith	ISP case opened; ETA 1 hour.	2025-02-14 14:45
23	18	Ticket 18: Appliance issue reported	3	Charlie Brown	Elevator vendor en route.	2025-03-22 09:30
29	24	Ticket Apr: pest control needed unit 8C	3	Charlie Brown	Elevator vendor en route.	2025-03-22 09:30
24	19	Ticket 19: Noise complaint reported	4	Diana Ross	Scheduled pest control for tomorrow.	2025-04-18 12:00
30	25	Ticket Jun: fire alarm test request	4	Diana Ross	Scheduled pest control for tomorrow.	2025-04-18 12:00

Tables Used: Comments, Tickets, Users.

Result Grid						
Filter Rows:						
Export:						
Wrap Cell Content:						
comment_id	ticket_id	ticket_description	user_id	full_name	comment_text	comment_date
1	1	Ticket 1: Plumbing issue reported	2	Bob Smith	Comment on ticket 1: Issue acknowledged.	2025-05-13 18:44
2	2	Ticket 2: Lift issue reported	3	Charlie Brown	Comment on ticket 2: Technician assigned.	2025-05-13 18:44
3	3	Ticket 3: Electrical issue reported	4	Diana Ross	Comment on ticket 3: Awaiting parts delivery.	2025-05-13 18:44
4	4	Ticket 4: HVAC issue reported	5	Edward Turner	Comment on ticket 4: Scheduled inspection.	2025-05-13 18:44
5	5	Ticket 5: Internet issue reported	6	Fiona Lee	Comment on ticket 5: Issue under review.	2025-05-13 18:44
6	6	Ticket 6: Garbage collection issue reported	7	George Martin	Comment on ticket 6: Replacement parts ordered.	2025-05-13 18:44
7	7	Ticket 7: Pest control issue reported	8	Hannah Davis	Comment on ticket 7: Follow-up required.	2025-05-13 18:44
8	8	Ticket 8: Structural issue reported	9	Ian Wright	Comment on ticket 8: Site visited by maintenance.	2025-05-13 18:44
9	9	Ticket 9: Security issue reported	10	Julia Roberts	Comment on ticket 9: Security notified.	2025-05-13 18:44

Query 8 – SLA Performance per Issue Category

Purpose: Measure compliance with SLA deadlines.

Details: Calculates the percentage of tickets resolved within SLA for each issue category.

Tables Used: Tickets, Ticket_Resolutions, Issue_Categories.

Result Grid				
Filter Rows:				
Export:				
Wrap Cell Content:				
category_name	total_resolved	resolved_within_sla	percent_within_sla	
Plumbing Issue	1	1	100.00	
Lift Issue	1	1	100.00	
Electrical Issue	1	1	100.00	
HVAC Issue	1	1	100.00	
Internet Issue	1	1	100.00	
Garbage Collection	1	1	100.00	
Pest Control	1	1	100.00	
Structural Issue	1	1	100.00	
Security Issue	1	1	100.00	

Query 9 – User Ticket Summary

Purpose: Provide performance summary per user.

Details: Displays total tickets raised, how many are closed, and the user's average resolution time for closed tickets.

Tables Used: Users, Tickets, Ticket_Status, Ticket_Resolutions.

Result Grid				
Filter Rows:				
Export:				
Wrap Cell Content:				
user_id	full_name	total_tickets	closed_tickets	avg_resolution_time_hours
1	Alice Johnson	3	2	4827.0000
2	Bob Smith	3	1	-842.0000
3	Charlie Brown	3	1	-858.0000
4	Diana Ross	3	1	-649.0000
5	Edward Turner	3	1	-1134.0000
6	Fiona Lee	3	1	-1312.0000
16	Patricia Clark	2	2	-1479.5000
17	Quincy Jones	2	2	-1058.0000

Query 10 – Monthly Ticket Trend

Purpose: Track ticket volume trends.

Details: Shows the number of tickets created each month for trend analysis.

Tables Used: Tickets.

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
month	tickets_created			
2025-01	2			
2025-02	2			
2025-03	2			
2025-04	2			
2025-05	20			
2025-06	2			
2025-07	2			

Stored Procedures

Below are 5 stored procedures implemented in the StarPrime database, with a brief explanation of each:

1. spCreateTicket

This procedure inserts a new record into the Tickets table. It captures user ID, property ID, issue category, description, SLA details, and optionally assigned agent or service provider. It helps standardize ticket creation logic and reduce manual errors.

Result Grid		Filter Rows:	Export:	Wrap Cell Content:		
	Procedure	sql_mode	Create Procedure	character_set_client	collation_connection	Database Collation
▶	spCreateTicket	STRICT_TRANS_TABLES,NO_ENGINE_SUBSTIT...	CREATE DEFINER='root'@'localhost' PROCEDURE...	utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900_ai_ci

2. spUpdateTicketStatus

Updates the status of a ticket (e.g., Open → In Progress → Closed) and records the change in the Audit_Log table. This ensures all updates are traceable and promotes transparency in ticket handling.

	property_name	total_requests
▶	Sunset Apartments	1
	Maple Grove	1
	Greenfield Estates	1
	Ocean View	1
	Mountain Retreat	1
	City Center Lofts	1

3. spAssignTicket

Assigns a support agent to a specific ticket and logs the assignment in the Audit_Log. It formalizes ticket delegation and tracks who is responsible for resolution.

	property_name	total_requests
	Willow Creek	1
	Forest Edge	1
	Skyline View	1
	Sunny Meadows	1
	Riverbend Estates	1
▶	Hilltop Residences	1

4. spEscalateTicket

Escalates a ticket's priority in a predefined sequence (Low → Medium → High → Urgent). Useful for handling aging or unresolved tickets by increasing their urgency automatically.

	property_name	total_requests
	Lakeside Villas	1
	Pine Hill Estates	1
	Urban Heights	1
	Coastal Retreat	1
	Desert Oasis	1
	Cedar Grove	1

5. spCloseTicket

Updates a ticket's status to 'Closed' and inserts a corresponding resolution record in the Ticket_Resolutions table. Also logs the action in Audit_Log. Helps enforce SLA and ensure every closure is documented with a solution.

102 Closed

Functions.

Below are five user-defined functions created in the StarPrime database. These were designed as functions (instead of procedures) because they return single, computed values and can be easily reused within queries, views, or other logic.

1. f_getTicketAge(ticket_id)

Returns the age of a ticket in hours from the time it was created. This helps in identifying overdue tickets or measuring resolution efficiency.

	ticket_id	created_at	ticket_age_in_hours
	1	2025-05-13 23:22:21	0
	2	2025-05-13 23:22:21	0

2. `f_getTicketPriority(ticket_id)`

Fetches the current priority level (Low, Medium, High, Urgent) of a specific ticket. Useful in SLA tracking, reporting, or escalation checks inside queries.

	ticket_id	priority
▶	1	Low
	2	Medium
	3	High
	4	Urgent
	5	Low
	6	Medium

3. `f_getUserFullName(user_id)`

Returns the full name of a user based on their user ID. This simplifies joins or report generation where user readability is important.

	ticket_id	assigned_to
▶	1	Alice Johnson
	2	Bob Smith
	3	Charlie Brown
	4	Diana Ross
	5	Edward Turner
	6	Fiona Lee

4. `f_getIssueCategory(category_id)`

Retrieves the name of the issue category (e.g., Plumbing, Electrical) for a given category ID. Used in dashboards or audit logs for better context.

category_name	ticket_count
Plumbing Issue	1
Lift Issue	1
Electrical Issue	1
HVAC Issue	1
Internet Issue	1
Garbage Collection	1

5. `f_timeRemainingForSLA(ticket_id)`
Calculates the remaining time (in hours) until a ticket breaches its SLA deadline. This function powers dashboards or alerts for proactive SLA compliance.

ticket_id	hours_remaining
3	23
7	23
11	23
15	23
19	23

Why Functions?

These were designed as functions because:

- They return single scalar values (like a name, priority, or number).
- They can be used in SELECT statements or embedded inside complex queries.
- They help modularize logic and avoid repetitive subqueries, improving performance and readability.

Triggers

1. `tr_before_insert_ticket`:
 - a. Table: Tickets
 - b. Event: BEFORE INSERT
 - c. Purpose: Ensures that the `sla_due` date is always later than the ticket's created at timestamp.
 - d. Why: Prevents invalid or illogical data during ticket creation.
2. `tr_after_update_ticket`
 - a. Table: Tickets

- b. Event: AFTER UPDATE
 - c. Purpose: When a ticket's priority is changed, the change is recorded in the Audit_Log table.
 - d. Why: Maintains a reliable audit trail for priority modifications.
- 3. tr_after_insert_resolution
 - a. Table: Ticket_Resolutions
 - b. Event: AFTER INSERT
 - c. Purpose: Automatically updates the ticket's status to 'Closed' once a resolution is recorded.
 - d. Why: Keeps status in sync with resolution entries without requiring manual updates.
- 4. tr_log_user_creation (Optional example if you added one)
 - a. Table: Users
 - b. Event: AFTER INSERT
 - c. Purpose: Could log when a new user is added or send a notification. (If not present, this can be removed.)
- 5. tr_notify_on_assignment (Optional/future scope)
 - a. Table: Tickets
 - b. Event: AFTER UPDATE
 - c. Purpose: Intended to send alerts when a support agent is assigned to a ticket. (Placeholder for enhancement.)

Data Security

1. Role-Based Access Control (RBAC):

- a. The User_Roles table defines different roles such as tenant, admin, support agent, and manager.
- b. Sensitive actions (like modifying tickets, assigning agents, or viewing all properties) are restricted based on role permissions.

2. Foreign Key Constraints with Cascading:

- a. Cascading actions (ON DELETE CASCADE or SET NULL) ensure that unauthorized or orphaned data cannot exist, maintaining data consistency and preventing leakage.
- 3. Sensitive Data Handling:
 - a. User passwords are stored in a column (password) intended to be encrypted (e.g., hashed before insertion in a real implementation).
 - b. Audit_Log tracks all sensitive ticket changes, enabling traceability and accountability.
- 4. Triggers for Automatic Logging:
 - a. Triggers like tr_after_update_ticket and tr_after_insert_resolution record who made changes and when, deterring unauthorized or accidental updates.
- 5. Data Integrity through Constraints:
 - a. Constraints such as NOT NULL, UNIQUE, and data-type validations reduce the risk of data corruption and injection.
- 6. Input Validation via Stored Procedures:
 - a. Stored procedures help control and validate the logic during operations like ticket creation, updates, or escalation, protecting against inconsistent or malicious inputs.

Implementation of AI / AI Agent in our Project and areas

To enhance the functionality and intelligence of our StarPrime property and ticket management system, we propose the integration of AI-based agents in the following areas:

- 1. AI-Powered Ticket Categorization:
 - a. Use Natural Language Processing (NLP) models to automatically categorize support tickets based on the description field.

- b. This reduces manual errors and accelerates response times by directly assigning issues to the correct category (e.g., Plumbing, Electrical).
- 2. Chatbot for Tenant Support:
 - a. Implement a conversational AI chatbot to assist tenants in raising tickets, checking ticket status, and retrieving FAQs.
 - b. The chatbot can be integrated with WhatsApp, Web UI, or mobile apps, providing 24/7 support.
- 3. SLA Breach Prediction:
 - a. Use historical data and machine learning models to predict which tickets are at high risk of missing their SLA deadlines.
 - b. This allows proactive alerts and prioritization by support teams.
- 4. Auto Assignment of Tickets:
 - a. An AI model can suggest or automatically assign tickets to the most suitable agent based on past performance, category expertise, and availability.
- 5. Sentiment Analysis on Comments/Feedback:
 - a. Analyze user comments and feedback using AI to detect dissatisfaction or frustration.
 - b. This helps the management intervene early and improve service quality.
- 6. Recommendation Engine for Resolutions:
 - a. Based on similar past tickets, AI can recommend likely resolutions or steps to fix a problem, assisting junior agents in faster ticket handling.

Enclosure:

YouTube Video Link:

<https://www.youtube.com/watch?v=xwNCO6V7xVU>



GitHub Link:

https://github.com/ShashankChandra97/DatabaseProject_G12

Disclaimer: The above GitHub link is to a private project, Access has been given to

Professor Kannan Srikanth

If you think you need access to check this GitHub repository, please send [Shashank Chandra](#) an email requesting access which will be considered upon brief identification.

StarPrime – Database Project Documentary

Course Name: **BUAN 6320.501 - Database Foundations for Business Analytics**

Group Members: Chowdhury, Nurul Ahad, Shashank Chandra, Sonawane, Sakshi Santosh, Yang, Yi-Hen, Vincent Sangala

Table of Contents

1. Project Overview

The StarPrime database project aims to streamline the submission, tracking, and management of maintenance requests for apartment residents. It addresses common issues such as delayed responses, communication gaps, and inefficiencies in handling repair tasks. The centralized system enhances service delivery, improves tracking and automates technician assignment based on workload and expertise.

2. Project Goals

- Enable residents to efficiently submit maintenance requests.
- Automate technician assignment based on request type and workload.
- Provide real-time tracking for residents.
- Generate maintenance analytics and reports to improve performance.
- Improve resident satisfaction through enhanced visibility and faster resolutions.

3. Deliverables

- Relational Database Design with tables, keys, and constraints.
- SQL Queries for request tracking, technician performance, and analytics.
- Stored Procedures for request creation, assignment, and status updates.
- Stored Functions for workload and performance calculations.
- Triggers for automated notifications and request status updates.
- ERD (Entity Relationship Diagram).
- Analytics Reports on maintenance trends and efficiency.
- Final Documentation.
- 7-minute Video Demonstration of the system.

4. Scope Definition

In Scope:

- Database Development
- Maintenance Request Tracking
- Automated Technician Assignment
- Status Notifications
- Role-Based Access Control
- Report Generation

Out of Scope:

- User Interface Development
- Integration with third-party software

5. Weekly Discussions & Contributions

- Project started on February 25.
- Created and updated the Project Charter.
- Weekly discussions held covering project goals, scope, and deliverables.
- Developed shared GitHub repository for collaborative code management.
- Group members contributed scripts, functions, procedures, and queries.
- Merged and tested scripts collaboratively.
- Created and finalized complex SQL queries.
- Submitted completed SQL script by due date.
- Held a successful team presentation.
- Completed a detailed project write-up and final in-class presentation.
- Produced a project video showcasing StarPrime functionality.

StarPrime Project Meeting Summary

Date	Discussion Summary
Feb 25	Group created and project finalized. Initial planning and kickoff.
Mar 2	Completed project scope. Discussed project overview. Held polling and submitted the project charter.
Mar 9	Discussed triggers and addition of functions.
Mar 11	Defined tables for StarPrime DB. Divided tasks and updated SQL scripts.
Mar 12	Call to submit the script.
Mar 15	Call to finalize table structures.
Mar 18	Call to run and debug scripts together.
Mar 19	GitHub walkthrough and setup demo.
Mar 21	Worked on GitHub pull requests.
Mar 22	Call to finalize GitHub. Merged everyone's code.
Mar 30	Class presentation.
Mar 31	Call to test final integrated script.
Apr 8	Started stored functions, procedures, and triggers.
Apr 12	Reviewed and finalized database code.
Apr 15	Completed final version of the SQL script.
Apr 20	Received and reviewed project write-up template.
Apr 22	Completed project documentation write-up.
Apr 24	Team discussion on in-class presentation format.

Apr 29	Added AI and security sections to the script. Updated complex queries. Gave final class presentation.
May 6	Started recording final StarPrime video.
May 12	Final video presentation completed.

6. Challenges Faced

- Ensuring referential integrity across all tables.
- Merging multiple code contributions in GitHub.
- Creating optimized and complex queries collaboratively.
- Managing progress and syncing development across all team members.

7. Final Outcomes

- A fully functional relational database supporting maintenance request lifecycle.
- Demonstrated automation, tracking, reporting, and workload distribution.
- Developed high-quality queries, triggers, procedures, and functions.
- Delivered documentation, code, presentation, and video as per project requirements.

8. Conclusion

The StarPrime project demonstrated effective team collaboration, technical skill application, and strategic planning. The solution addresses real-world apartment maintenance challenges and enhances efficiency through smart database design.

9. Appendix

- SQL Scripts
- ER Diagram
- Screenshots of outputs
- GitHub Repo Link
- Presentation Slides
- Video Link